

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No: 92200133037

Introduction

InterviewBot is a web-based application designed to simulate job interviews using AI-driven question generation, text-to-speech (TTS), speech recognition, and a 3D animated interviewer avatar. The system allows users to submit personal details and resumes, participate in a tailored interview, receive real-time feedback, record sessions, and download performance summaries as PDFs. Built with a Flask backend and HTML/JavaScript frontend, it integrates external APIs (Google Gemini for question/feedback generation, ElevenLabs for TTS) and uses Tesseract for resume OCR. The implementation adheres to best coding practices, ensures robust functionality, and achieves seamless component integration, aligning with the project's objectives to provide an immersive, reliable interview practice platform.

This section details the code structure, implementation details, testing procedures, and setup instructions, accompanied by a system architecture diagram. The project evolved iteratively across versions, starting with basic question-asking (V1), adding depth through follow-up questions based on previous answers (V2), and culminating in the final version with immediate reviews for negative responses (e.g., "repeat question") and comprehensive end-of-interview summaries (V3). The source code is hosted on a GitHub repository (link to be provided post-Render deployment).

Module Descriptions

- **app.py:** Core Flask application defining endpoints for user data submission (/submit_user_data), question generation (/generate_questions), TTS (/tts), feedback (/generate_answer_feedback), answer storage (/store_answers), and PDF generation (/generate_pdf). It includes error handling and logging.
- **Frontend HTML Files:**
 - index.html: Form for user details and resume upload, with webcam photo capture.
 - rules.html: Displays guidelines and consent button.
 - interview.html: Implements 3D avatar (Three.js), webcam feed, speech recognition (Web Speech API), and TTS playback.
 - summary.html: Displays performance metrics and enables PDF downloads.
 - screen.html: Test page for screen recording (MediaRecorder).
- **Static Assets:**
 - style.css: Responsive design with clamp-based font sizing.
 - scripts.js: Handles frontend logic (form submission, API calls, avatar animation).
- **JSON Files:** Temporary storage for user data (user.json), answers (answers.json), and sample report (interview_report.json).

Version Evolution

The implementation progressed iteratively, with each version building on the previous to enhance realism and user engagement:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No: 92200133037

- **Version 1 (V1):** Focused on core functionality where the system only asked new, independent questions. The `/generate_questions` endpoint generated a fixed set of 5 questions based solely on the job description and resume, without considering prior responses. This was implemented using a basic Gemini prompt in `app.py`, ensuring initial stability and question relevance.
- **Version 2 (V2):** Updated to enable dynamic, in-depth questioning like a real interview. The endpoint was enhanced to support `is_follow_up=True`, generating questions based on the user's previous answer (`previous_answer` parameter). This added progression in difficulty and context-awareness, probing deeper (e.g., "Can you elaborate on your Python project?"). Changes were tracked in Git commits (e.g., "Add follow-up logic to `generate_questions`").
- **Version 3 (Final):** Retained all prior functionality but introduced immediate reviews for negative or repetitive user inputs (e.g., "repeat question" triggers re-asking without new feedback, while vague answers get constructive reviews via `/generate_answer_feedback`). At the interview's end, a comprehensive summary report is generated (`/generate_summary`), including scores, strengths, weaknesses, and feedback stored in `interview_report.json`. This version emphasizes user-friendly error handling and actionable insights.

Version Control

The project uses Git for version control, hosted on GitHub (link to be provided). Commit history reflects the evolution:

- V1 commits: Initial setup and basic question generation.
- V2 commits: Follow-up question logic and prompt refinements.
- V3 commits: Feedback integration, summary endpoint, and bug fixes.

Implementation Details

Key Technologies

- **Backend:** Flask 2.3.3 (Python) for API endpoints, Gunicorn 22.0.0 for deployment.
- **Frontend:** HTML5, CSS3, JavaScript (Three.js for 3D avatar, Web Speech API for speech recognition, MediaRecorder for recording).
- **External APIs:**
 - **Google Gemini:** Generates tailored interview questions and feedback.
 - **ElevenLabs:** Provides high-quality TTS, with gTTS as a fallback.
- **OCR:** Tesseract 5.3.0 and Poppler 22.12 for resume text extraction.
- **PDF Generation:** pdflatex (TeX Live 2023) for compiling LaTeX reports.
- **Other Libraries:** Pillow (image processing), pandas (OCR data analysis), requests (API calls).

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No: 92200133037

3.2 Key Algorithms and Protocols

- **Resume Name Extraction (V1-V3):**
 - Algorithm: Combines Tesseract OCR with regex and text height analysis to extract candidate names.
 - Process: Converts resume to text, uses regex for direct matches, and height filtering for candidates.
 -
- **Question Generation (Evolved from V1 to V3):**
 - V1: Basic Gemini prompt for independent questions.
 - V2: Added previous_answer for follow-ups, ensuring depth (e.g., scenario-based probes).
 - V3: Incorporated review logic—if user says "repeat question," re-ask without new feedback.
 -
- **TTS Fallback (V1-V3):**
 - Primary: ElevenLabs for realistic voice.
 - Fallback: gTTS for API failures.
 -
- **PDF Generation (V3):**
 - Converts JSON report to LaTeX, compiles to PDF.
 - Sanitizes inputs for security.
- **HTTP Protocol:** RESTful APIs with JSON payloads.

Error Handling and Input Validation

- **Backend:**
 - Validates inputs (e.g., email regex, file types).
 - Retry logic for API calls (3 attempts).
 - Sanitizes LaTeX to prevent injection.
- **Frontend:**
 - Required fields in forms, error alerts for failed API calls.
- **Evolution:** V3 added handling for negative user inputs (e.g., "repeat question" triggers review).

Coding Standards

- **Python (PEP 8):** 4-space indentation, descriptive variables, comments for complex logic.
 - **JavaScript:** Modular functions with comments.
 - **Modularity:** Separate endpoints/files for each feature.
-

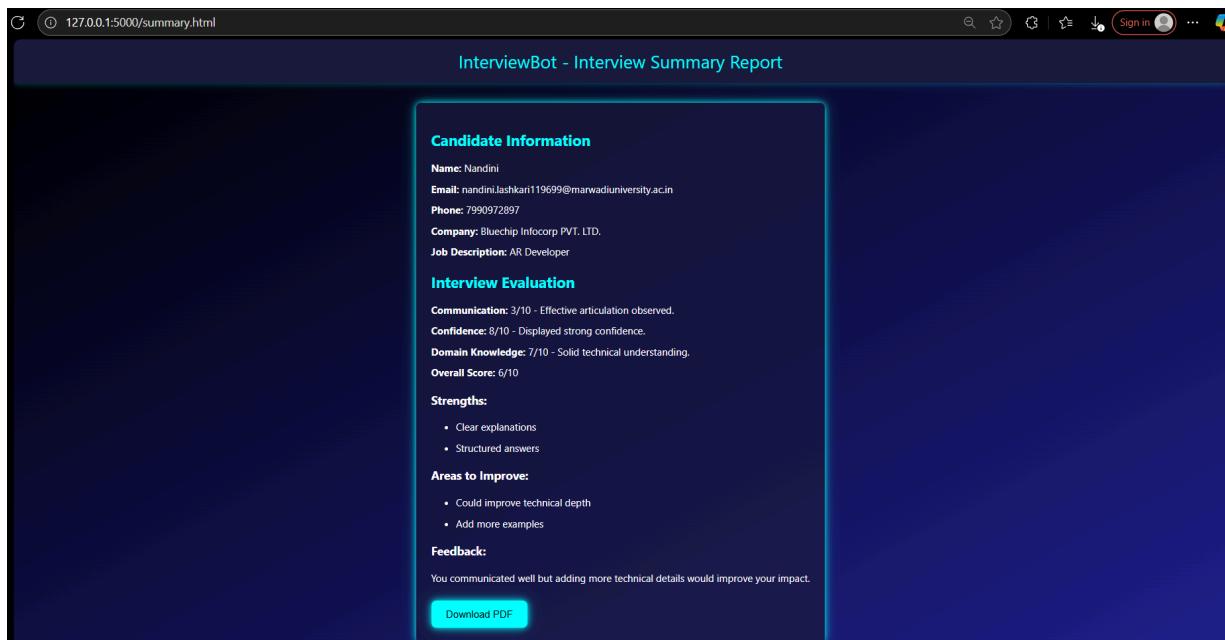
 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No: 92200133037

Functionality

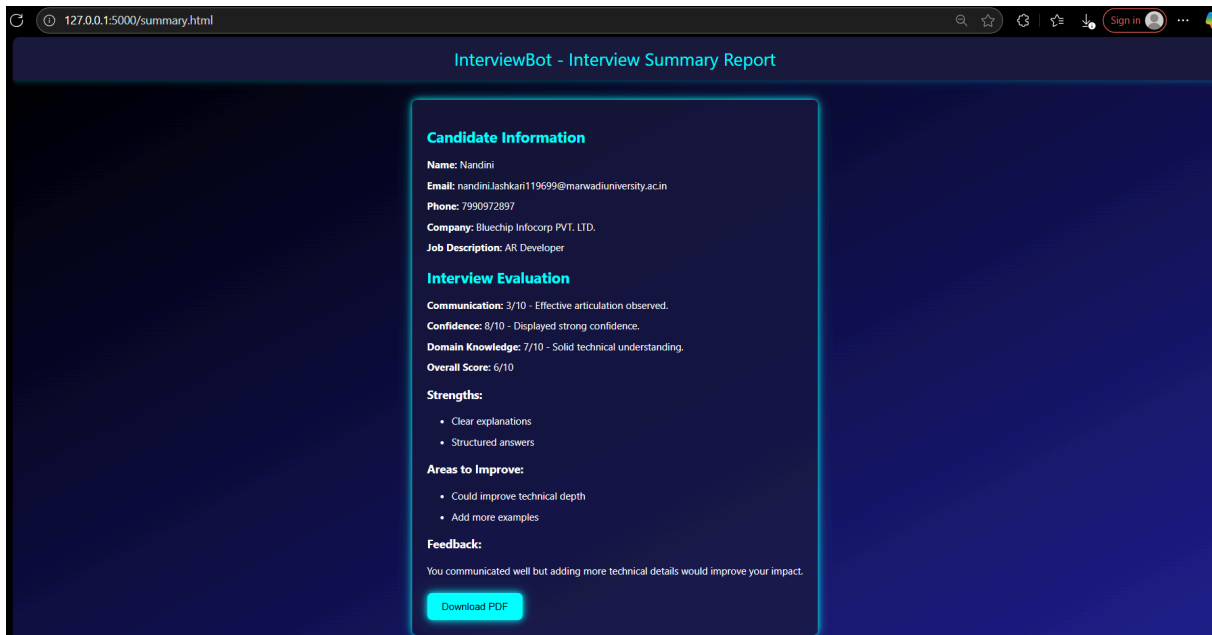
InterviewBot meets all functional requirements:

1. **User Submission:** Processes resumes and stores data (V1-V3).
2. **Interview Simulation:** Generates/asks questions, evolving from static (V1) to dynamic follow-ups (V2) with reviews (V3).
3. **Feedback:** Immediate for answers, with end summary (V3).
4. **Recording:** Video capture in interview.html.
5. **PDF Export:** Downloads reports.

The system performs reliably, with V3's review mechanism enhancing realism (e.g., "That's okay, let's move on" for vague answers).



 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No:92200133037



Integration Across Components

Component Interactions

- **Frontend ↔ Backend:** JS Fetch calls Flask endpoints.
- **Backend ↔ JSON:** Reads/writes for persistence.
- **Backend ↔ APIs:** Direct calls to Gemini/ElevenLabs.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Capstone Project	Aim: Implementation	
	Date: 25/09/25	Enrolment No: 92200133037

- **Evolution:** V2 integrated previous answers for questions; V3 added review logic in /generate_answer_feedback.

Integration Testing

Manual tests verified flows:

- Form → JSON → Questions → Answers → Summary.
- Results: All passed, confirming integration.

